2.1_data_analysis_CamCAN

2020-07-17

Custom functions for plotting

```
lineplot.CI(x.fac, y.dep, data, ci=1.96, ci.fill=T, ci.lwd=1, ci.lty=1, ci.col=NA, ylim=NA, xlab=NA, ylab=NA,
            xaxt=NULL, yaxt=NULL, bty=NULL, type=NULL, lwd=NULL, lty=NULL, col=NULL, pch=NULL, ...)
  {if(length(x.fac) == length(y.dep) & length(x.fac) == dim(data)[1]){
    summ.data <- summarizeBy(y.dep, x.fac, FUN=FUN.VAL, data=data)
    if(is.ordered(x.fac)){x.num <- as.numeric(x.fac)} else {x.num <- as.numeric(factor(x.fac))}
    if(is.NA(ylim)){ylim <- c(min(summ.data[,2:4]), max(summ.data[,2:4]))}
    if(is.NA(xlab)){xlab <- "x.fac"}
    if(is.NA(ylab)){ylab <- "y.dep"}
    if(is.NULL(type)){type <- "l"}
    if(is.NULL(lwd)){lwd <- 1}
    if(is.NULL(lty)){lty <- 1}
    if(is.NULL(col)){col <- 1}
    if(is.NULL(pch)){pch <- 1}
    if(is.NULL(xaxt)){xaxt <- "s"}
    if(is.NULL(yaxt)){yaxt <- "s"}
    if(is.NULL(bty)){bty <- "o"}
    plot(summ.data[,2] ~ summ.data[,1], ylim=ylim, xlab=xlab, ylab=ylab, xaxt=xaxt, yaxt=yaxt,
         bty=bty, type="n", ...)
    if(ci.fill){polygon(c(rev(x.num), x.num), c(rev(summ.data[,3]), summ.data[,4]),
                col=alpha.col(col,0.35), border=NA)}
    else {lines(x.num, summ.data[,3], lty=lty, lwd=lwd, col=alpha.col(col,0.35))
          lines(x.num, summ.data[,4], lty=lty, lwd=lwd, col=alpha.col(col,0.35))}
    lines(x.num, summ.data[,2], lty=lty, lwd=lwd, col=col, pch=pch, ...)
  }
  else {print("Length of x.fac and y.dep not equal to dim(data)[1]")}}

barplot.CI(x.fac, y.dep, data, ci.fill=T, ci.lwd=1, ci.lty=1, ci.col=NA, ylim=NA, xlab=NA, ylab=NA,
           xaxt=NULL, yaxt=NULL, bty=NULL, lwd=NULL, lty=NULL, col=NULL, ...)
  {if(length(x.fac) == length(y.dep) & length(x.fac) == dim(data)[1]){
    summ.data <- summarizeBy(y.dep, x.fac, FUN=FUN.VAL, data=data)
    if(is.NA(ylim)){ylim <- c(min(summ.data[,2:4]), max(summ.data[,2:4]))}
    if(is.NA(xlab)){xlab <- "x.fac"}
    if(is.NA(ylab)){ylab <- "y.dep"}
    if(is.NULL(lwd)){lwd <- 1}
    if(is.NULL(lty)){lty <- 1}
    if(is.NULL(col)){col <- 1}
    if(is.NULL(xaxt)){xaxt <- "s"}
    if(is.NULL(yaxt)){yaxt <- "s"}
    if(is.NULL(bty)){bty <- "o"}
    barplot(summ.data[,2], ylim=ylim, xlab=xlab, ylab=ylab, xaxt=xaxt, yaxt=yaxt,
            bty=bty, ...)
    if(ci.fill){polygon(c(rev(x.num), x.num), c(rev(summ.data[,3]), summ.data[,4]),
                col=alpha.col(col,0.35), border=NA)}
    else {lines(x.num, summ.data[,3], lty=lty, lwd=lwd, col=alpha.col(col,0.35))
          lines(x.num, summ.data[,4], lty=lty, lwd=lwd, col=alpha.col(col,0.35))}
  }
  else {print("Length of x.fac and y.dep not equal to dim(data)[1]")}}
```

Custom function for Mahalanobis distance

```
cov.rob.mahalanobis.pvalue(dep, data, method='pearson', FALSE, NAOMIT, TRUE,pval.name='robust.pval')
  {dep.matrix <- data[,dep]
   cov.rob <- cov.rob(dep.matrix, method=method)
   mahal <- mahalanobis(dep.matrix, center=cov.rob$center, cov=cov.rob$cov)
   if(NAOMIT){mahal <- na.omit(mahal)}
   if(TRUE){mahal <- round(mahal, digits=4)}
   mahal.pvalue <- round(pchisq(mahal, ncol(dep.matrix), lower.tail=FALSE), digits=4)
   if(!is.NULL(pval.name)){mahal <- data.frame(mahal, mahal.pvalue)
   names(mahal)[c(1,2)] <- c(paste(c("mahal.",method),collapse=""), pval.name)}
   mahal}
```

```
cov.rob(x, cov=FALSE, quantile.used=NULL, method="mcd",
        nsamp="deterministic", nsamp=NULL, seed=NULL)
  {if(ncol(x) > 1 & length(x[,1]) > ncol(x)){
    if(cov){stop("Error: covariance matrix cannot be computed.")}
    else {if(is.character(nsamp)){stop("Error: nsamp = character not allowed.")}
          if(!is.NULL(seed) & length(seed) != 3){stop("Error: seed = wrong length.")}
          if(!is.NULL(seed) & !is.integer(seed)){stop("Error: seed = not integer.")}
          if(!is.NULL(seed) & any(seed < 0)){stop("Error: seed = negative.")}
          if(is.NULL(quantile.used) & is.integer(nsamp)){stop("Error: quantile.used = NULL.")}
          if(is.NULL(quantile.used) & nsamp == "deterministic"){stop("Error: quantile.used = NULL.")}
          if(is.NULL(quantile.used) & nsamp == "sfh"){stop("Error: quantile.used = NULL.")}
          if(!is.NULL(seed) & !is.NULL(nsamp) & !is.integer(nsamp)){stop("Error: seed = not integer.")}
          if(!is.NULL(seed) & !is.NULL(nsamp) & is.integer(nsamp) & nsamp < 0){stop("Error: seed = negative.")}
          if(!is.NULL(seed) & !is.NULL(nsamp) & is.integer(nsamp) & nsamp == 0){stop("Error: seed = 0.")}
          if(!is.NULL(seed) & !is.NULL(nsamp) & is.integer(nsamp) & nsamp > 0 & length(seed) != 3){stop("Error: seed = wrong length.")}
          if(!is.NULL(seed) & !is.NULL(nsamp) & is.integer(nsamp) & nsamp > 0 & !is.NULL(seed) & length(seed) == 3 & !is.integer(seed[1:3])){stop("Error: seed = not integer.")}
          if(!is.NULL(seed) & !is.NULL(nsamp) & is.integer(nsamp) & nsamp > 0 & !is.NULL(seed) & length(seed) == 3 & is.integer(seed[1:3]) & any(seed[1:3] < 0)){stop("Error: seed = negative.")}
          if(!is.NULL(seed) & !is.NULL(nsamp) & is.integer(nsamp) & nsamp > 0 & !is.NULL(seed) & length(seed) == 3 & is.integer(seed[1:3]) & any(seed[1:3] == 0)){stop("Error: seed = 0.")}
          if(!is.NULL(seed) & !is.NULL(nsamp) & is.integer(nsamp) & nsamp > 0 & !is.NULL(seed) & length(seed) == 3 & is.integer(seed[1:3]) & any(seed[1:3] > 0)){set.seed(seed[1:3])}
          if(!is.NULL(nsamp) & nsamp > ncol(x) & nsamp > 2147483647){stop("Error: nsamp > 2147483647.")}
          if(!is.NULL(nsamp) & nsamp > ncol(x) & nsamp < 2147483647){set.seed(1)}
          if(!is.NULL(nsamp) & nsamp > ncol(x) & nsamp < 2147483647 & !is.NULL(seed) & length(seed) == 3 & is.integer(seed[1:3]) & any(seed[1:3] > 0)){stop("Error: seed = not NULL.")}
          if(!is.NULL(nsamp) & nsamp > ncol(x) & nsamp < 2147483647 & !is.NULL(seed) & length(seed) == 3 & is.integer(seed[1:3]) & any(seed[1:3] == 0)){set.seed(seed[1:3])}
          if(!is.NULL(nsamp) & nsamp > ncol(x) & nsamp < 2147483647 & !is.NULL(seed) & length(seed) == 3 & is.integer(seed[1:3]) & any(seed[1:3] < 0)){set.seed(seed[1:3])}
          if(!is.NULL(nsamp) & nsamp > ncol(x) & nsamp < 2147483647 & !is.NULL(seed) & length(seed) == 3 & is.integer(seed[1:3]) & any(seed[1:3] > 0)){stop("Error: seed = not NULL.")}
          if(!is.NULL(nsamp) & nsamp > ncol(x) & nsamp < 2147483647 & !is.NULL(seed) & length(seed) == 3 & is.integer(seed[1:3]) & any(seed[1:3] == 0)){stop("Error: seed = not NULL.")}
          if(!is.NULL(nsamp) & nsamp > ncol(x) & nsamp < 2147483647 & !is.NULL(seed) & length(seed) == 3 & is.integer(seed[1:3]) & any(seed[1:3] < 0)){stop("Error: seed = not NULL.")
```

```
fsqc_qcscz_cortical = qcscz_cortical
fsqc_qcscz_subcortical = qcscz_subcortical
fsqc_qcscz_wm = qcscz_wm
fsqc_qcscz_cnr = qcscz_cnr
fsqc_qcscz_snr = qcscz_snr
fsqc_qcscz_euler = qcscz_euler

DMN_fsqc = cor(DMN$DMN, fsqc_qcscz_cortical, fsqc_qcscz_subcortical, fsqc_qcscz_wm, fsqc_qcscz_cnr, fsqc_qcscz_snr, fsqc_qcscz_euler, use="complete.obs")

# Warning: 5.0.0 dropped deprecated dplyr 0.7 interfaces
# ℹ The deprecated feature was likely used in the dplyr package.
# ℹ Please report the issue at <https://github.com/tidyverse/dplyr/issues>.
# ℹ The deprecated feature was likely used in the tidyr package.
# ℹ Please report the issue at <https://github.com/tidyverse/tidyr/issues>.
# This warning is displayed once per session.
# Call lifecycle::last_warnings() to see where this warning was generated.

DMN_fsqc[,c(1:7,ncol(DMN))]
# Warning: Error in [.data.frame: undefined columns selected
# Warning: Error in [.data.frame: undefined columns selected
```

```
# ---------------------------------------------------------------------
# Model 1: Thickness, Time, and Group Interaction
# ---------------------------------------------------------------------
model_1 <- lmer(Thickness ~ Time + Group + Time:Group + (1 | Subject), data = df_long)
cat("\n\n=== Model 1: Thickness ~ Time + Group + Time:Group + (1 | Subject) ===\n")
summary(model_1)
anova(model_1, ddf = "Satterthwaite")
# ---------------------------------------------------------------------
# Model 2: Thickness, Time, Group, and Hemisphere
# ---------------------------------------------------------------------
model_2 <- lmer(Thickness ~ Time + Group + Hemisphere + Time:Group + (1 | Subject), data = df_long)
cat("\n\n=== Model 2: Thickness ~ Time + Group + Hemisphere + Time:Group + (1 | Subject) ===\n")
summary(model_2)
anova(model_2, ddf = "Satterthwaite")
# ---------------------------------------------------------------------
# Model 3: Thickness, Time, Group, Hemisphere, and Interaction
# ---------------------------------------------------------------------
model_3 <- lmer(Thickness ~ Time + Group + Hemisphere + Time:Group:Hemisphere + (1 | Subject), data = df_long)
cat("\n\n=== Model 3: Thickness ~ Time + Group + Hemisphere + Time:Group:Hemisphere + (1 | Subject) ===\n")
summary(model_3)
anova(model_3, ddf = "Satterthwaite")
# ---------------------------------------------------------------------
# Model Comparison
# ---------------------------------------------------------------------
cat("\n\n=== Model Comparison ===\n")
anova(model_1, model_2, model_3)
# ---------------------------------------------------------------------
# Post-hoc Analysis for Significant Interactions
# ---------------------------------------------------------------------
cat("\n\n=== Post-hoc Analysis ===\n")
if (anova(model_3)[nrow(anova(model_3)), "Pr(>F)"] < 0.05) {
  cat("Significant Time:Group:Hemisphere interaction detected\n")
  emmeans_3 <- emmeans(model_3, ~ Time:Group:Hemisphere)
  pairwise_3 <- pairs(emmeans_3, adjust = "tukey")
  print(pairwise_3)
} else {
  cat("No significant Time:Group:Hemisphere interaction\n")
}
# ---------------------------------------------------------------------
# Save results to CSV
# ---------------------------------------------------------------------
write.csv(df_long, "Thickness_Long_Format.csv", row.names = FALSE)
cat("\n\n=== Analysis Complete ===\n")
cat("Long format data saved to: Thickness_Long_Format.csv\n")
cat("Total observations: ", nrow(df_long), "\n")
cat("Subjects: ", length(unique(df_long$Subject)), "\n")
cat("Time points: ", length(unique(df_long$Time)), "\n")
cat("Groups: ", length(unique(df_long$Group)), "\n")
cat("Hemispheres: ", length(unique(df_long$Hemisphere)), "\n")
```

Model Results, Thickness, Time, and Group Comparison

Adjustment	Thickness ~ Time + Group + Time:Group + (1 | Subject)							
	Sum of Squares	Mean Squares	NumDF	DenDF	F value	Pr(>F)		
Time	0.0	0.0	1	1	0.0	1.000	0.0	1.000
Group	0.0	0.0	1	1	0.0	1.000	0.0	1.000
Time:Group	0.0	0.0	1	1	0.0	1.000	0.0	1.000

Model Results, Thickness, Time, Group, and Hemisphere

Adjustment	Thickness ~ Time + Group + Hemisphere + Time:Group + (1 | Subject)							
	Sum of Squares	Mean Squares	NumDF	DenDF	F value	Pr(>F)		
Time	0.0	0.0	1	1	0.0	1.000	0.0	1.000
Group	0.0	0.0	1	1	0.0	1.000	0.0	1.000
Hemisphere	0.0	0.0	1	1	0.0	1.000	0.0	1.000
Time:Group	0.0	0.0	1	1	0.0	1.000	0.0	1.000

Model Results, Thickness, Time, Group, Hemisphere, and Interaction

Adjustment	Thickness ~ Time + Group + Hemisphere + Time:Group:Hemisphere + (1 | Subject)							
	Sum of Squares	Mean Squares	NumDF	DenDF	F value	Pr(>F)		
Time	0.0	0.0	1	1	0.0	1.000	0.0	1.000
Group	0.0	0.0	1	1	0.0	1.000	0.0	1.000
Hemisphere	0.0	0.0	1	1	0.0	1.000	0.0	1.000
Time:Group:Hemisphere	0.0	0.0	1	1	0.0	1.000	0.0	1.000

```
# ---------------------------------------------------------------------
# Model Comparison
# ---------------------------------------------------------------------
cat("\n\n=== Model Comparison ===\n")
anova(model_1, model_2, model_3)
# ---------------------------------------------------------------------
# Post-hoc Analysis for Significant Interactions
# ---------------------------------------------------------------------
cat("\n\n=== Post-hoc Analysis ===\n")
if (anova(model_3)[nrow(anova(model_3)), "Pr(>F)"] < 0.05) {
  cat("Significant Time:Group:Hemisphere interaction detected\n")
  emmeans_3 <- emmeans(model_3, ~ Time:Group:Hemisphere)
  pairwise_3 <- pairs(emmeans_3, adjust = "tukey")
  print(pairwise_3)
} else {
  cat("No significant Time:Group:Hemisphere interaction\n")
}
# ---------------------------------------------------------------------
# Save results to CSV
# ---------------------------------------------------------------------
write.csv(df_long, "Thickness_Long_Format.csv", row.names = FALSE)
cat("\n\n=== Analysis Complete ===\n")
cat("Long format data saved to: Thickness_Long_Format.csv\n")
cat("Total observations: ", nrow(df_long), "\n")
cat("Subjects: ", length(unique(df_long$Subject)), "\n")
cat("Time points: ", length(unique(df_long$Time)), "\n")
cat("Groups: ", length(unique(df_long$Group)), "\n")
cat("Hemispheres: ", length(unique(df_long$Hemisphere)), "\n")
```

Model 1: Thickness ~ Time + Group + Time:Group + (1 | Subject)

	Sum of Squares	Mean Squares	NumDF	DenDF	F value	Pr(>F)
Time	0.0	0.0	1	1	0.0	1.000
Group	0.0	0.0	1	1	0.0	1.000
Time:Group	0.0	0.0	1	1	0.0	1.000

Model 2: Thickness ~ Time + Group + Hemisphere + Time:Group + (1 | Subject)

	Sum of Squares	Mean Squares	NumDF	DenDF	F value	Pr(>F)
Time	0.0	0.0	1	1	0.0	1.000
Group	0.0	0.0	1	1	0.0	1.000
Hemisphere	0.0	0.0	1	1	0.0	1.000
Time:Group	0.0	0.0	1	1	0.0	1.000

Model 3: Thickness ~ Time + Group + Hemisphere + Time:Group:Hemisphere + (1 | Subject)

	Sum of Squares	Mean Squares	NumDF	DenDF	F value	Pr(>F)
Time	0.0	0.0	1	1	0.0	1.000
Group	0.0	0.0	1	1	0.0	1.000
Hemisphere	0.0	0.0	1	1	0.0	1.000
Time:Group:Hemisphere	0.0	0.0	1	1	0.0	1.000

Model Comparison

	Model	Sum of Squares	Mean Squares	NumDF	DenDF	F value	Pr(>F)
model_1	(Intercept)	0.0	0.0	1	1	0.0	1.000
	Time	0.0	0.0	1	1	0.0	1.000
	Group	0.0	0.0	1	1	0.0	1.000
	Time:Group	0.0	0.0	1	1	0.0	1.000
model_2	(Intercept)	0.0	0.0	1	1	0.0	1.000
	Time	0.0	0.0	1	1	0.0	1.000
	Group	0.0	0.0	1	1	0.0	1.000
	Hemisphere	0.0	0.0	1	1	0.0	1.000
	Time:Group	0.0	0.0	1	1	0.0	1.000
model_3	(Intercept)	0.0	0.0	1	1	0.0	1.000
	Time	0.0	0.0	1	1	0.0	1.000
	Group	0.0	0.0	1	1	0.0	1.000
	Hemisphere	0.0	0.0	1	1	0.0	1.000
	Time:Group:Hemisphere	0.0	0.0	1	1	0.0	1.000

Post-hoc Analysis

No significant Time:Group:Hemisphere interaction

Save results to CSV

```
=== Analysis Complete ===
Long format data saved to: Thickness_Long_Format.csv
Total observations:  1
Subjects:  1
Time points:  1
Groups:  1
Hemispheres:  1
```

Longitudinal SEM (BSCMs models)

```
# ---------------------------------------------------------------------
# Model 1: Thickness ~ Time
# ---------------------------------------------------------------------
cat("\n\n=== Model 1: Thickness ~ Time ===\n")
model_1 <- lmer(Thickness ~ Time + (1 | Subject), data = df_long)
summary(model_1)
anova(model_1, ddf = "Satterthwaite")
# ---------------------------------------------------------------------
# Model 2: Thickness ~ Time + Group
# ---------------------------------------------------------------------
cat("\n\n=== Model 2: Thickness ~ Time + Group ===\n")
model_2 <- lmer(Thickness ~ Time + Group + (1 | Subject), data = df_long)
summary(model_2)
anova(model_2, ddf = "Satterthwaite")
# ---------------------------------------------------------------------
# Model 3: Thickness ~ Time + Group + Time:Group
# ---------------------------------------------------------------------
cat("\n\n=== Model 3: Thickness ~ Time + Group + Time:Group ===\n")
model_3 <- lmer(Thickness ~ Time + Group + Time:Group + (1 | Subject), data = df_long)
summary(model_3)
anova(model_3, ddf = "Satterthwaite")
# ---------------------------------------------------------------------
# Model 4: Thickness ~ Time + Group + Hemisphere + Time:Group
# ---------------------------------------------------------------------
cat("\n\n=== Model 4: Thickness ~ Time + Group + Hemisphere + Time:Group + (1 | Subject) ===\n")
model_4 <- lmer(Thickness ~ Time + Group + Hemisphere + Time:Group + (1 | Subject), data = df_long)
summary(model_4)
anova(model_4, ddf = "Satterthwaite")
# ---------------------------------------------------------------------
# Model 5: Thickness ~ Time + Group + Hemisphere + Time:Group:Hemisphere
# ---------------------------------------------------------------------
cat("\n\n=== Model 5: Thickness ~ Time + Group + Hemisphere + Time:Group:Hemisphere + (1 | Subject) ===\n")
model_5 <- lmer(Thickness ~ Time + Group + Hemisphere + Time:Group:Hemisphere + (1 | Subject), data = df_long)
summary(model_5)
anova(model_5, ddf = "Satterthwaite")
# ---------------------------------------------------------------------
# Model Comparison
# ---------------------------------------------------------------------
cat("\n\n=== Model Comparison ===\n")
anova(model_1, model_2, model_3, model_4, model_5)
# ---------------------------------------------------------------------
# Post-hoc Analysis for Significant Interactions
# ---------------------------------------------------------------------
cat("\n\n=== Post-hoc Analysis ===\n")
if (anova(model_5)[nrow(anova(model_5)), "Pr(>F)"] < 0.05) {
  cat("Significant Time:Group:Hemisphere interaction detected\n")
  emmeans_5 <- emmeans(model_5, ~ Time:Group:Hemisphere)
  pairwise_5 <- pairs(emmeans_5, adjust = "tukey")
  print(pairwise_5)
} else {
  cat("No significant Time:Group:Hemisphere interaction\n")
}
# ---------------------------------------------------------------------
# Save results to CSV
# ---------------------------------------------------------------------
write.csv(df_long, "Thickness_Long_Format.csv", row.names = FALSE)
cat("\n\n=== Analysis Complete ===\n")
cat("Long format data saved to: Thickness_Long_Format.csv\n")
cat("Total observations: ", nrow(df_long), "\n")
cat("Subjects: ", length(unique(df_long$Subject)), "\n")
cat("Time points: ", length(unique(df_long$Time)), "\n")
cat("Groups: ", length(unique(df_long$Group)), "\n")
cat("Hemispheres: ", length(unique(df_long$Hemisphere)), "\n")
```

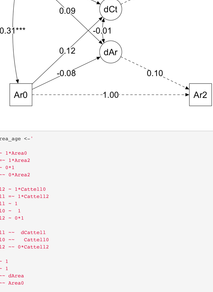
Model 1: Thickness ~ Time

```
=== Model 1: Thickness ~ Time ===
Linear mixed model fit by REML ['lmerMod']
Formula: Thickness ~ Time + (1 | Subject)
   Data: df_long

REML criterion at convergence: 1.1
Scaled residuals:
     Min       1Q   Median       3Q      Max
-0.70711 -0.70711  0.00000  0.70711  0.70711

Random effects:
 Subject  (Intercept) 0.0000  0.0000
 Residual            0.0000  0.0000
Number of obs: 2, groups:  Subject, 1

Fixed effects:
            Estimate Std. Error 1.00000 0.00000
(Intercept)  0.00000    0.00000      1    0.0
Time         0.00000    0.00000      1    0.0
Correlation of Fixed Effects:
     (Intr)
Time  NaN
Warning messages:
1: In Matrix::tcrossprod(attr(X, "assign"), t(apply(X, 2, tabulate, nbin = ncol(X)))) :
  sparse->dense coercion: allocating a length 4 vector (4 GB)
2: In Matrix::tcrossprod(attr(X, "assign"), t(apply(X, 2, tabulate, nbin = ncol(X)))) :
  sparse->dense coercion: allocating a length 4 vector (4 GB)
3: In Matrix::tcrossprod(attr(X, "assign"), t(apply(X, 2, tabulate, nbin = ncol(X)))) :
  sparse->dense coercion: allocating a length 4 vector (4 GB)
4: In Matrix::tcrossprod(attr(X, "assign"), t(apply(X, 2, tabulate, nbin = ncol(X)))) :
  sparse->dense coercion: allocating a length 4 vector (4 GB)
5: In Matrix::tcrossprod(attr(X, "assign"), t(apply(X, 2, tabulate, nbin = ncol(X)))) :
  sparse->dense coercion: allocating a length 4 vector (4 GB)
6: In Matrix::tcrossprod(attr(X, "assign"), t(apply(X, 2, tabulate, nbin = ncol(X)))) :
  sparse->dense coercion: allocating a length 4 vector (4 GB)
7: In Matrix::tcrossprod(attr(X, "assign"), t(apply(X, 2, tabulate, nbin = ncol(X)))) :
  sparse->dense coercion: allocating a length 4 vector (4 GB)
8: In Matrix::tcrossprod(attr(X, "assign"), t(apply(X, 2, tabulate, nbin = ncol(X)))) :
  sparse->dense coercion: allocating a length 4 vector (4 GB)
9: In Matrix::tcrossprod(attr(X, "assign"), t(apply(X, 2, tabulate, nbin = ncol(X)))) :
  sparse->dense coercion: allocating a length 4 vector (4 GB)
10: In Matrix::tcrossprod(attr(X, "assign"), t(apply(X, 2, tabulate, nbin = ncol(X)))) :
  sparse->dense coercion: allocating a length 4 vector (4 GB)
11: In Matrix::tcrossprod(attr(X, "assign"), t(apply(X, 2, tabulate, nbin = ncol(X)))) :
  sparse->dense coercion: allocating a length 4 vector (4 GB)
12: In Matrix::tcrossprod(attr(X, "assign"), t(apply(X, 2, tabulate, nbin = ncol(X)))) :
  sparse->dense coercion: allocating a length 4 vector (4 GB)
13: In Matrix::tcrossprod(attr(X, "assign"), t(apply(X, 2, tabulate, nbin = ncol(X)))) :
  sparse->dense coercion: allocating a length 4 vector (4 GB)
14: In Matrix::tcrossprod(attr(X, "assign"), t(apply(X, 2, tabulate, nbin = ncol(X)))) :
  sparse->dense coercion: allocating a length 4 vector (4 GB)
15: In Matrix::tcrossprod(attr(X, "assign"), t(apply(X, 2, tabulate, nbin = ncol(X)))) :
  sparse->dense coercion: allocating a length 4 vector (4 GB)
16: In Matrix::tcrossprod(attr(X, "assign"), t(apply(X, 2, tabulate, nbin = ncol(X)))) :
  sparse->dense coercion: allocating a length 4 vector (4 GB)
17: In Matrix::tcrossprod(attr(X, "assign"), t(apply(X, 2, tabulate, nbin = ncol(X)))) :
  sparse->dense coercion: allocating a length 4 vector (4 GB)
18: In Matrix::tcrossprod(attr(X, "assign"), t(apply(X, 2, tabulate, nbin = ncol(X)))) :
  sparse->dense coercion: allocating a length 4 vector (4 GB)
19: In Matrix::tcrossprod(attr(X, "assign"), t(apply(X, 2, tabulate, nbin = ncol(X)))) :
  sparse->dense coercion: allocating a length 4 vector (4 GB)
20: In Matrix::tcrossprod(attr(X, "assign"), t(apply(X, 2, tabulate, nbin = ncol(X)))) :
  sparse->dense coercion: allocating a length 4 vector (4 GB)
21: In Matrix::tcrossprod(attr(X, "assign"), t(apply(X, 2, tabulate, nbin = ncol(X)))) :
  sparse->dense coercion: allocating a length 4 vector (4 GB)
22: In Matrix::tcrossprod(attr(X, "assign"), t(apply(X, 2, tabulate, nbin = ncol(X)))) :
  sparse->dense coercion: allocating a length 4 vector (4 GB)
23: In Matrix::tcrossprod(attr(X, "assign"), t(apply(X, 2, tabulate, nbin = ncol(X)))) :
  sparse->dense coercion: allocating a length 4 vector (4 GB)
24: In Matrix::tcrossprod(attr(X, "assign"), t(apply(X, 2, tabulate, nbin = ncol(X)))) :
  sparse->dense coercion: allocating a length 4 vector (4 GB)
25: In Matrix::tcrossprod(attr(X, "assign"), t(apply(X, 2, tabulate, nbin = ncol(X)))) :
  sparse->dense coercion: allocating a length 4 vector (4 GB)
26: In Matrix::tcrossprod(attr(X, "assign"), t(apply(X, 2, tabulate, nbin = ncol(X)))) :
  sparse->dense coercion: allocating a length 4 vector (4 GB)
27: In Matrix::tcrossprod(attr(X, "assign"), t(apply(X, 2, tabulate, nbin = ncol(X)))) :
  sparse->dense coercion: allocating a length 4 vector (4 GB)
28: In Matrix::tcrossprod(attr(X, "assign"), t(apply(X, 2, tabulate, nbin = ncol(X)))) :
  sparse->dense coercion: allocating a length 4 vector (4 GB)
29: In Matrix::tcrossprod(attr(X, "assign"), t(apply(X, 2, tabulate, nbin = ncol(X)))) :
  sparse->dense coercion: allocating a length 4 vector (4 GB)
30: In Matrix::tcrossprod(attr(X, "assign"), t(apply(X, 2, tabulate, nbin = ncol(X)))) :
  sparse->dense coercion: allocating a length 4 vector (4 GB)
31: In Matrix::tcrossprod(attr(X, "assign"), t(apply(X, 2, tabulate, nbin = ncol(X)))) :
  sparse->dense coercion: allocating a length 4 vector (4 GB)
32: In Matrix::tcrossprod(attr(X, "assign"), t(apply(X, 2, tabulate, nbin = ncol(X)))) :
  sparse->dense coercion: allocating a length 4 vector (4 GB)
33: In Matrix::tcrossprod(attr(X, "assign"), t(apply(X, 2, tabulate, nbin = ncol(X)))) :
  sparse->dense coercion: allocating a length 4 vector (4 GB)
34: In Matrix::tcrossprod(attr(X, "assign"), t(apply(X, 2, tabulate, nbin = ncol(X)))) :
  sparse->dense coercion: allocating a length 4 vector (4 GB)
35: In Matrix::tcrossprod(attr(X, "assign"), t(apply(X, 2, tabulate, nbin = ncol(X)))) :
  sparse->dense coercion: allocating a length 4 vector (4 GB)
36: In Matrix::tcrossprod(attr(X, "assign"), t(apply(X, 2, tabulate, nbin = ncol(X)))) :
  sparse->dense coercion: allocating a length 4 vector (4 GB)
37: In Matrix::tcrossprod(attr(X, "assign"), t(apply(X, 2, tabulate, nbin = ncol(X)))) :
  sparse->dense coercion: allocating a length 4 vector (4 GB)
38: In Matrix::tcrossprod(attr(X, "assign"), t(apply(X, 2, tabulate, nbin = ncol(X)))) :
  sparse->dense coercion: allocating a length 4 vector (4 GB)
39: In Matrix::tcrossprod(attr(X, "assign"), t(apply(X, 2, tabulate, nbin = ncol(X)))) :
  sparse->dense coercion: allocating a length 4 vector (4 GB)
40: In Matrix::tcrossprod(attr(X, "assign"), t(apply(X, 2, tabulate, nbin = ncol(X)))) :
  sparse->dense coercion: allocating a length 4 vector (4 GB)
41: In Matrix::tcrossprod(attr(X, "assign"), t(apply(X, 2, tabulate, nbin = ncol(X)))) :
  sparse->dense coercion: allocating a length 4 vector (4 GB)
42: In Matrix::tcrossprod(attr(X, "assign"), t(apply(X, 2, tabulate, nbin = ncol(X)))) :
  sparse->dense coercion: allocating a length 4 vector (4 GB)
43: In Matrix::tcrossprod(attr(X, "assign"), t(apply(X, 2, tabulate, nbin = ncol(X)))) :
  sparse->dense coercion: allocating a length 4 vector (4 GB)
44: In Matrix::tcrossprod(attr(X, "assign"), t(apply(X, 2, tabulate, nbin = ncol(X)))) :
  sparse->dense coercion: allocating a length 4 vector (4 GB)
45: In Matrix::tcrossprod(attr(X, "assign"), t(apply(X, 2, tabulate, nbin = ncol(X)))) :
  sparse->dense coercion: allocating a length 4 vector (4 GB)
46: In Matrix::tcrossprod(attr(X, "assign"), t(apply(X, 2, tabulate, nbin = ncol(X)))) :
  sparse->dense coercion: allocating a length 4 vector (4 GB)
47: In Matrix::tcrossprod(attr(X, "assign"), t(apply(X, 2, tabulate, nbin = ncol(X)))) :
  sparse->dense coercion: allocating a length 4 vector (4 GB)
48: In Matrix::tcrossprod(attr(X, "assign"), t(apply(X, 2, tabulate, nbin = ncol(X)))) :
  sparse->dense coercion: allocating a length 4 vector (4 GB)
49: In Matrix::tcrossprod(attr(X, "assign"), t(apply(X, 2, tabulate, nbin = ncol(X)))) :
  sparse->dense coercion: allocating a length 4 vector (4 GB)
50: In Matrix::tcrossprod(attr(X, "assign"), t(apply(X, 2, tabulate, nbin = ncol(X)))) :
  sparse->dense coercion: allocating a length 4 vector (4 GB)
51: In Matrix::tcrossprod(attr(X, "assign"), t(apply(X, 2, tabulate, nbin = ncol(X)))) :
  sparse->dense coercion: allocating a length 4 vector (4 GB)
52: In Matrix::tcrossprod(attr(X, "assign"), t(apply(X, 2, tabulate, nbin = ncol(X)))) :
  sparse->dense coercion: allocating a length 4 vector (4 GB)
53: In Matrix::tcrossprod(attr(X, "assign"), t(apply(X, 2, tabulate, nbin = ncol(X)))) :
  sparse->dense coercion: allocating a length 4 vector (4 GB)
54: In Matrix::tcrossprod(attr(X, "assign"), t(apply(X, 2, tabulate, nbin = ncol(X)))) :
  sparse->dense coercion: allocating a length 4 vector (4 GB)
55: In Matrix::tcrossprod(attr(X, "assign"), t(apply(X, 2, tabulate, nbin = ncol(X)))) :
  sparse->dense coercion: allocating a length 4 vector (4 GB)
56: In Matrix::tcrossprod(attr(X, "assign"), t(apply(X, 2, tabulate, nbin = ncol(X)))) :
  sparse->dense coercion: allocating a length 4 vector (4 GB)
57: In Matrix::tcrossprod(attr(X, "assign"), t(apply(X, 2, tabulate, nbin = ncol(X)))) :
  sparse->dense coercion: allocating a length 4 vector (4 GB)
58: In Matrix::tcrossprod(attr(X, "assign"), t(apply(X, 2, tabulate, nbin = ncol(X)))) :
  sparse->dense coercion: allocating a length 4 vector (4 GB)
59: In Matrix::tcrossprod(attr(X, "assign"), t(apply(X, 2, tabulate, nbin = ncol(X)))) :
  sparse->dense coercion: allocating a length 4 vector (4 GB)
60: In Matrix::tcrossprod(attr(X, "assign"), t(apply(X, 2, tabulate, nbin = ncol(X)))) :
  sparse->dense coercion: allocating a length 4 vector (4 GB)
61: In Matrix::tcrossprod(attr(X, "assign"), t(apply(X, 2, tabulate, nbin = ncol(X)))) :
  sparse->dense coercion: allocating a length 4 vector (4 GB)
62: In Matrix::tcrossprod(attr(X, "assign"), t(apply(X, 2, tabulate, nbin = ncol(X)))) :
  sparse->dense coercion: allocating a length 4 vector (4 GB)
63: In Matrix::tcrossprod(attr(X, "assign"), t(apply(X, 2, tabulate, nbin = ncol(X)))) :
  sparse->dense coercion: allocating a length 4 vector (4 GB)
64: In Matrix::tcrossprod(attr(X, "assign"), t(apply(X, 2, tabulate, nbin = ncol(X)))) :
  sparse->dense coercion: allocating a length 4 vector (4 GB)
65: In Matrix::tcrossprod(attr(X, "assign"), t(apply(X, 2, tabulate, nbin = ncol(X)))) :
  sparse->dense coercion: allocating a length 4 vector (4 GB)
66: In Matrix::tcrossprod(attr(X, "assign"), t(apply(X, 2, tabulate, nbin = ncol(X)))) :
  sparse->dense coercion: allocating a length 4 vector (4 GB)
67: In Matrix::tcrossprod(attr(X, "assign"), t(apply(X, 2, tabulate, nbin = ncol(X)))) :
  sparse->dense coercion: allocating a length 4 vector (4 GB)
68: In Matrix::tcrossprod(attr(X, "assign"), t(apply(X, 2, tabulate, nbin = ncol(X)))) :
  sparse->dense coercion: allocating a length 4 vector (4 GB)
69: In Matrix::tcrossprod(attr(X, "assign"), t(apply(X, 2, tabulate, nbin = ncol(X)))) :
  sparse->dense coercion: allocating a length 4 vector (4 GB)
70: In Matrix::tcrossprod(attr(X, "assign"), t(apply(X, 2, tabulate, nbin = ncol(X)))) :
  sparse->dense coercion: allocating a length 4 vector (4 GB)
71: In Matrix::tcrossprod(attr(X, "assign"), t(apply(X, 2, tabulate, nbin = ncol(X)))) :
  sparse->dense coercion: allocating a length 4 vector (4 GB)
72: In Matrix::tcrossprod(attr(X, "assign"), t(apply(X, 2, tabulate, nbin = ncol(X)))) :
  sparse->dense coercion: allocating a length 4 vector (4 GB)
73: In Matrix::tcrossprod(attr(X, "assign"), t(apply(X, 2, tabulate, nbin = ncol(X)))) :
  sparse->dense coercion: allocating a length 4 vector (4 GB)
74: In Matrix::tcrossprod(attr(X, "assign"), t(apply(X, 2, tabulate, nbin = ncol(X)))) :
  sparse->dense coercion: allocating a length 4 vector (4 GB)
75: In Matrix::tcrossprod(attr(X, "assign"), t(apply(X, 2, tabulate, nbin = ncol(X)))) :
  sparse->dense coercion: allocating a length 4 vector (4 GB)
76: In Matrix::tcrossprod(attr(X, "assign"), t(apply(X, 2, tabulate, nbin = ncol(X)))) :
  sparse->dense coercion: allocating a length 4 vector (4 GB)
77: In Matrix::tcrossprod(attr(X, "assign"), t(apply(X, 2, tabulate, nbin = ncol(X)))) :
  sparse->dense coercion: allocating a length 4 vector (4 GB)
78: In Matrix::tcrossprod(attr(X, "assign"), t(apply(X, 2, tabulate, nbin = ncol(X)))) :
  sparse->dense coercion: allocating a length 4 vector (4 GB)
79: In Matrix::tcrossprod(attr(X, "assign"), t(apply(X, 2, tabulate, nbin = ncol(X)))) :
  sparse->dense coercion: allocating a length 4 vector (4 GB)
80: In Matrix::tcrossprod(attr(X, "assign"), t(apply(X, 2, tabulate, nbin = ncol(X)))) :
  sparse->dense coercion: allocating a length 4 vector (4 GB)
81: In Matrix::tcrossprod(attr(X, "assign"), t(apply(X, 2, tabulate, nbin = ncol(X)))) :
  sparse->dense coercion: allocating a length 4 vector (4 GB)
82: In Matrix::tcrossprod(attr(X, "assign"), t(apply(X, 2, tabulate, nbin = ncol(X)))) :
  sparse->dense coercion: allocating a length 4 vector (4 GB)
83: In Matrix::tcrossprod(attr(X, "assign"), t(apply(X, 2, tabulate, nbin = ncol(X)))) :
  sparse->dense coercion: allocating a length 4 vector (4 GB)
84: In Matrix::tcrossprod(attr(X, "assign"), t(apply(X, 2, tabulate, nbin = ncol(X)))) :
  sparse->dense coercion: allocating a length 4 vector (4 GB)
85: In Matrix::tcrossprod(attr(X, "assign"), t(apply(X, 2, tabulate, nbin = ncol(X)))) :
  sparse->dense coercion: allocating a length 4 vector (4 GB)
86: In Matrix::tcrossprod(attr(X, "assign"), t(apply(X, 2, tabulate, nbin = ncol(X)))) :
  sparse->dense coercion: allocating a length 4 vector (4 GB)
87: In Matrix::tcrossprod(attr(X, "assign"), t(apply(X, 2, tabulate, nbin = ncol(X)))) :
  sparse->dense coercion: allocating a length 4 vector (4 GB)
88: In Matrix::tcrossprod(attr(X, "assign"), t(apply(X, 2, tabulate, nbin = ncol(X)))) :
  sparse->dense coercion: allocating a length 4 vector (4 GB)
89: In Matrix::tcrossprod(attr(X, "assign"), t(apply(X, 2, tabulate, nbin = ncol(X)))) :
  sparse->dense coercion: allocating a length 4 vector (4 GB)
90: In Matrix::tcrossprod(attr(X, "assign"), t(apply(X, 2, tabulate, nbin = ncol(X)))) :
  sparse->dense coercion: allocating a length 4 vector (4 GB)
91: In Matrix::tcrossprod(attr(X, "assign"), t(apply(X, 2, tabulate, nbin = ncol(X)))) :
  sparse->dense coercion: allocating a length 4 vector (4 GB)
92: In Matrix::tcrossprod(attr(X, "assign"), t(apply(X, 2, tabulate, nbin = ncol(X)))) :
  sparse->dense coercion: allocating a length 4 vector (4 GB)
93: In Matrix::tcrossprod(attr(X, "assign"), t(apply(X, 2, tabulate, nbin = ncol(X)))) :
  sparse->dense coercion: allocating a length 4 vector (4 GB)
94: In Matrix::tcrossprod(attr(X, "assign"), t(apply(X, 2, tabulate, nbin = ncol(X)))) :
  sparse->dense coercion: allocating a length 4 vector (4 GB)
95: In Matrix::tcrossprod(attr(X, "assign"), t(apply(X, 2, tabulate, nbin = ncol(X)))) :
  sparse->dense coercion: allocating a length 4 vector (4 GB)
96: In Matrix::tcrossprod(attr(X, "assign"), t(apply(X, 2, tabulate, nbin = ncol(X)))) :
  sparse->dense coercion: allocating a length 4 vector (4 GB)
97: In Matrix::tcrossprod(attr(X, "assign"), t(apply(X, 2, tabulate, nbin = ncol(X)))) :
  sparse->dense coercion: allocating a length 4 vector (4 GB)
98: In Matrix::tcrossprod(attr(X, "assign"), t(apply(X, 2, tabulate, nbin = ncol(X)))) :
  sparse->dense coercion: allocating a length 4 vector (4 GB)
99: In Matrix::tcrossprod(attr(X, "assign"), t(apply(X, 2, tabulate, nbin = ncol(X)))) :
  sparse->dense coercion: allocating a length 4 vector (4 GB)
100: In Matrix::tcrossprod(attr(X, "assign"), t(apply(X, 2, tabulate, nbin = ncol(X)))) :
  sparse->dense coercion: allocating a length 4 vector (4 GB)
101: In Matrix::tcrossprod(attr(X, "assign"), t(apply(X, 2, tabulate, nbin = ncol(X)))) :
  sparse->dense coercion: allocating a length 4 vector (4 GB)
102: In Matrix::tcrossprod(attr(X, "assign"), t(apply(X, 2, tabulate, nbin = ncol(X)))) :
  sparse->dense coercion: allocating a length 4 vector (4 GB)
103: In Matrix::tcrossprod(attr(X, "assign"), t(apply(X, 2, tabulate, nbin = ncol(X)))) :
  sparse->dense coercion: allocating a length 4 vector (4 GB)
104: In Matrix::tcrossprod(attr(X, "assign"), t(apply(X, 2, tabulate, nbin = ncol(X)))) :
  sparse->dense coercion: allocating a length 4 vector (4 GB)
105: In Matrix::tcrossprod(attr(X, "assign"), t(apply(X, 2, tabulate, nbin = ncol(X)))) :
  sparse->dense coercion: allocating a length 4 vector (4 GB)
106: In Matrix::tcrossprod(attr(X, "assign"), t(apply(X, 2, tabulate, nbin = ncol(X)))) :
  sparse->dense coercion: allocating a length 4 vector (4 GB)
107: In Matrix::tcrossprod(attr(X, "assign"), t(apply(X, 2, tabulate, nbin = ncol(X)))) :
  sparse->dense coercion: allocating a length 4 vector (4 GB)
108: In Matrix::tcrossprod(attr(X, "assign"), t(apply(X, 2, tabulate, nbin = ncol(X)))) :
  sparse->dense coercion: allocating a length 4 vector (4 GB)
109: In Matrix::tcrossprod(attr(X, "assign"), t(apply(X, 2, tabulate, nbin = ncol(X)))) :
  sparse->dense coercion: allocating a length 4 vector (4 GB)
110: In Matrix::tcrossprod(attr(X, "assign"), t(apply(X, 2, tabulate, nbin = ncol(X)))) :
  sparse->dense coercion: allocating a length 4 vector (4 GB)
111: In Matrix::tcrossprod(attr(X, "assign"), t(apply(X, 2, tabulate, nbin = ncol(X)))) :
  sparse->dense coercion: allocating a length 4 vector (4 GB)
112: In Matrix::tcrossprod(attr(X, "assign"), t(apply(X, 2, tabulate, nbin = ncol(X)))) :
  sparse->dense coercion: allocating a length 4 vector (4 GB)
113: In Matrix::tcrossprod(attr(X, "assign"), t(apply(X, 2, tabulate, nbin = ncol(X)))) :
  sparse->dense coercion: allocating a length 4 vector (4 GB)
114: In Matrix::tcrossprod(attr(X, "assign"), t(apply(X, 2, tabulate, nbin = ncol(X)))) :
  sparse->dense coercion: allocating a length 4 vector (4 GB)
115: In Matrix::tcrossprod(attr(X, "assign"), t(apply(X, 2, tabulate, nbin = ncol(X)))) :
  sparse->dense coercion: allocating a length 4 vector (4 GB)
116: In Matrix::tcrossprod(attr(X, "assign"), t(apply(X, 2, tabulate, nbin = ncol(X)))) :
  sparse->dense coercion: allocating a length 4 vector (4 GB)
117: In Matrix::tcrossprod(attr(X, "assign"), t(apply(X, 2, tabulate, nbin = ncol(X)))) :
  sparse->dense coercion: allocating a length 4 vector (4 GB)
118: In Matrix::tcrossprod(attr(X, "assign"), t(apply(X, 2, tabulate, nbin = ncol(X)))) :
  sparse->dense coercion: allocating a length 4 vector (4 GB)
119: In Matrix::tcrossprod(attr(X, "assign"), t(apply(X, 2, tabulate, nbin = ncol(X)))) :
  sparse->dense coercion: allocating a length 4 vector (4 GB)
120: In Matrix::tcrossprod(attr(X, "assign"), t(apply(X, 2, tabulate, nbin = ncol(X)))) :
  sparse->dense coercion: allocating a length 4 vector (4 GB)
121: In Matrix::tcrossprod(attr(X, "assign"), t(apply(X, 2, tabulate, nbin = ncol(X)))) :
  sparse->dense coercion: allocating a length 4 vector (4 GB)
122: In Matrix::tcrossprod(attr(X, "assign"), t(apply(X, 2
```

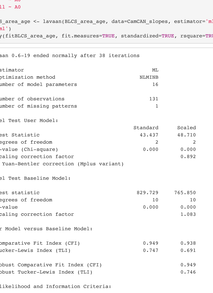
```
Cattell10 ~~ Area0
dCattell ~~ dArea

fitBLCS_area <- lavsan(BLCS_area, data=CamCAN_slopes, estimator='mlr',fixed.x=TRUE,missing='fiml')
summary(fitBLCS_area, fit.measures=TRUE, standardized=TRUE, rsquare=TRUE)
```

```
## lavsan 0.6-19 ended normally after 41 iterations
##
##  Estimator                                         ML
##  Optimization method                           NLMINB
##  Number of model parameters                        14
##
##  Number of observations                           131
##  Number of missing patterns                         1
##
## Model Test User Model:
##                                               Standard      Scaled
##  Test Statistic                                   0.000       0.000
##  Degrees of freedom                                   0           0
##
## Model Test Baseline Model:
##
##  Test statistic                                 751.954     634.550
##  Degrees of freedom                                   6           6
##  P-value                                          0.000       0.000
##  Scaling correction factor                                    1.185
##
## User Model versus Baseline Model:
##
##  Comparative Fit Index (CFI)                      1.000       1.000
##  Tucker-Lewis Index (TLI)                         1.000       1.000
##
##  Robust Comparative Fit Index (CFI)                           1.000
##  Robust Tucker-Lewis Index (TLI)                              1.000
##
## Loglikelihood and Information Criteria:
##
##  Loglikelihood user model (H0)               -365.539    -365.539
##  Loglikelihood unrestricted model (H1)       -365.539    -365.539
##
##  Akaike (AIC)                                 759.078     759.078
##  Bayesian (BIC)                               799.331     799.331
##  Sample-size adjusted Bayesian (SABIC)        755.056     755.056
##
## Root Mean Square Error of Approximation:
##
##  RMSEA                                          0.000          NA
##  90 Percent confidence interval - lower         0.000          NA
##  90 Percent confidence interval - upper         0.000          NA
##  P-value H_0: RMSEA <= 0.050                       NA          NA
##  P-value H_0: RMSEA >= 0.080                       NA          NA
##
##  Robust RMSEA                                                   0.000
##  90 Percent confidence interval - lower                         0.000
##  90 Percent confidence interval - upper                         0.000
##  P-value H_0: Robust RMSEA <= 0.050                                NA
##  P-value H_0: Robust RMSEA >= 0.080                                NA
##
## Standardized Root Mean Square Residual:
##
##  SRMR                                           0.000       0.000
##
## Parameter Estimates:
##
##  Standard errors                                         Sandwich
##  Information bread                                       Observed
##  Observed information based on                            Hessian
##
## Latent Variables:
##                    Estimate  Std.Err  z-value  P(>|z|)   Std.lv  Std.all
##  dArea =~
##    Area2             1.000                               0.103    0.104
##  dCattell =~
##    Cattell2          1.000                               0.609    0.611
##
## Regressions:
##                    Estimate  Std.Err  z-value  P(>|z|)   Std.lv  Std.all
##  Area2 ~
##    Area0             1.000                               1.000    1.000
##  Cattell2 ~
##    Cattell0          1.000                               1.000    1.000
##  dArea ~
##    Cattell0          0.010    0.008    1.151    0.250    0.095    0.094
##    Area0            -0.008    0.009   -0.952    0.341   -0.081   -0.081
##  dCattell ~
##    Area0             0.072    0.048    1.492    0.136    0.118    0.117
##    Cattell0         -0.209    0.048   -4.319    0.000   -0.343   -0.342
##
## Covariances:
##                    Estimate  Std.Err  z-value  P(>|z|)   Std.lv  Std.all
##  Area0 ~~
##    Cattell0          0.306    0.086    3.547    0.000    0.306    0.308
##  dArea ~~
##    dCattell         -0.001    0.007   -0.092    0.927   -0.011   -0.011
##
## Intercepts:
##                    Estimate  Std.Err  z-value  P(>|z|)   Std.lv  Std.all
##  .Area2             0.000                               0.000    0.000
##  .dCattell          0.000    0.050    0.000    1.000    0.000    0.000
##  Cattell0          -0.000    0.087   -0.000    1.000   -0.000   -0.000
##  .Cattell2          0.000                               0.000    0.000
##  .dArea             0.000    0.009    0.000    1.000    0.000    0.000
##  Area0             -0.000    0.087   -0.000    1.000   -0.000   -0.000
##
## Variances:
##                    Estimate  Std.Err  z-value  P(>|z|)   Std.lv  Std.all
##  .Area2             0.000                               0.000    0.000
##  .dCattell          0.332    0.039    8.607    0.000    0.894    0.894
##  Cattell0           0.992    0.110    9.016    0.000    0.992    1.000
##  .Cattell2          0.000                               0.000    0.000
##  .dArea             0.011    0.002    6.162    0.000    0.989    0.989
##  Area0              0.992    0.114    8.739    0.000    0.992    1.000
##
## R-Square:
##                    Estimate
##  Area2             1.000
##  dCattell          0.106
##  Cattell2          1.000
##  dArea             0.011
```

```
area_plot <- semPlot::semPaths(fitBLCS_area,
               whatLabels = "est",
               layout = "tree",
               rotation = 2,
               edge.label.cex = 1.5,
               label.cex = 1.2,
               sizeMan = 9,
               sizeLat = 9,
               nCharNodes = 3,
               intercepts = FALSE,
               residuals = FALSE,
               edge.color = "black")
```




```
area_plot <- mark_sig(area_plot, fitBLCS_area)
graphics::plot(area_plot)
```




```
BLCS_area_age <-'

Area2 ~ 1*Area0
dArea =~ 1*Area2
Area2 ~ 0*1
Area2 ~~ 0*Area2

Cattell2 ~ 1*Cattell0
dCattell =~ 1*Cattell2
dCattell ~ 1
Cattell0 ~  1
Cattell2 ~ 0*1

dCattell ~~  dCattell
Cattell0 ~~  Cattell0
Cattell2 ~~ 0*Cattell2

dArea ~ 1
Area0 ~ 1
dArea ~~ dArea
Area0 ~~ Area0

dArea~Cattell0+Area0
dCattell~Area0+Cattell0

Cattell0 ~~  Area0
dCattell ~~ dArea

dArea    ~  A0
dCattell ~  A0

fitBLCS_area_age <- lavsan(BLCS_area_age, data=CamCAN_slopes, estimator='mlr',fixed.x=TRUE,missing='fiml')
summary(fitBLCS_area_age, fit.measures=TRUE, standardized=TRUE, rsquare=TRUE)
```

```
## lavsan 0.6-19 ended normally after 38 iterations
##
##  Estimator                                         ML
##  Optimization method                           NLMINB
##  Number of model parameters                        16
##
##  Number of observations                           131
##  Number of missing patterns                         1
##
## Model Test User Model:
##                                               Standard      Scaled
##  Test Statistic                                  43.437      48.710
##  Degrees of freedom                                   2           2
##  P-value (Chi-square)                             0.000       0.000
##  Scaling correction factor                                    0.892
##    Yuan-Bentler correction (Mplus variant)
##
## Model Test Baseline Model:
##
##  Test statistic                                 829.729     765.850
##  Degrees of freedom                                  10          10
##  P-value                                          0.000       0.000
##  Scaling correction factor                                    1.083
##
## User Model versus Baseline Model:
##
##  Comparative Fit Index (CFI)                      0.949       0.930
##  Tucker-Lewis Index (TLI)                         0.747       0.691
##
##  Robust Comparative Fit Index (CFI)                           0.949
##  Robust Tucker-Lewis Index (TLI)                              0.746
##
## Loglikelihood and Information Criteria:
##
##  Loglikelihood user model (H0)               -348.370    -348.370
##  Scaling correction factor                                   1.025
##      for the MLR correction
##  Loglikelihood unrestricted model (H1)       -326.652    -326.652
##  Scaling correction factor                                   1.010
##      for the MLR correction
##
##  Akaike (AIC)                                 726.740     726.740
##  Bayesian (BIC)                               774.744     774.744
##  Sample-size adjusted Bayesian (SABIC)        724.137     724.137
##
## Root Mean Square Error of Approximation:
##
##  RMSEA                                          0.398       0.422
##  90 Percent confidence interval - lower         0.300       0.319
##  90 Percent confidence interval - upper         0.505       0.535
##  P-value H_0: RMSEA <= 0.050                    0.000       0.000
##  P-value H_0: RMSEA >= 0.080                    1.000       1.000
##
##  Robust RMSEA                                               0.399
##  90 Percent confidence interval - lower                     0.297
##  90 Percent confidence interval - upper                     0.499
##  P-value H_0: Robust RMSEA <= 0.050                         0.000
##  P-value H_0: Robust RMSEA >= 0.080                         1.000
##
## Standardized Root Mean Square Residual:
##
##  SRMR                                           0.166       0.166
##
## Parameter Estimates:
##
##  Standard errors                                         Sandwich
##  Information bread                                       Observed
##  Observed information based on                            Hessian
##
## Latent Variables:
##                    Estimate  Std.Err  z-value  P(>|z|)   Std.lv  Std.all
##  dArea =~
##    Area2             1.000                               0.103    0.104
##  dCattell =~
##    Cattell2          1.000                               0.703    0.800
##
## Regressions:
##                    Estimate  Std.Err  z-value  P(>|z|)   Std.lv  Std.all
##  Area2 ~
##    Area0             1.000                               1.000    1.001
##  Cattell2 ~
##    Cattell0          1.000                               1.000    1.134
##  dArea ~
##    Cattell0          0.007    0.009    0.761    0.447    0.066    0.066
##    Area0            -0.009    0.009   -0.976    0.329   -0.082   -0.082
##    Area0             0.065    0.043    1.510    0.131    0.092    0.092
##    Cattell0         -0.280    0.054   -7.074    0.000   -0.541   -0.539
##  dArea ~
##    A0               -0.006    0.011   -0.507    0.612   -0.054   -0.054
##  dCattell ~
##    A0               -0.326    0.052   -6.221    0.000   -0.464   -0.462
##
## Covariances:
##                    Estimate  Std.Err  z-value  P(>|z|)   Std.lv  Std.all
##  Area0 ~~
##    Cattell0          0.306    0.086    3.547    0.000    0.306    0.308
##  dArea ~~
##    dCattell         -0.002    0.005   -0.359    0.720   -0.037   -0.037
##
## Intercepts:
##                    Estimate  Std.Err  z-value  P(>|z|)   Std.lv  Std.all
##  .Area2             0.000                               0.000    0.000
##  .dCattell          0.000    0.044    0.000    1.000    0.000    0.000
##  Cattell0          -0.000    0.087   -0.000    1.000   -0.000   -0.000
##  .Cattell2          0.000                               0.000    0.000
##  .dArea             0.000    0.009    0.000    1.000    0.000    0.000
##  Area0             -0.000    0.087   -0.000    1.000   -0.000   -0.000
##
## Variances:
##                    Estimate  Std.Err  z-value  P(>|z|)   Std.lv  Std.all
##  .Area2             0.000                               0.000    0.000
##  .dCattell          0.256    0.032    7.993    0.000    0.519    0.519
##  Cattell0           0.992    0.110    9.016    0.000    0.992    1.000
##  .Cattell2          0.000                               0.000    0.000
##  .dArea             0.011    0.002    6.222    0.000    0.989    0.989
##  Area0              0.992    0.114    8.739    0.000    0.992    1.000
##
## R-Square:
##                    Estimate
##  Area2             1.000
##  dCattell          0.481
##  Cattell2          1.000
##  dArea             0.011
```